

Segment-5

Inheritance

Inheritance:

The capability of a class to derive properties and characteristics from another classes is called **Inheritance**. Inheritance is one of the most important features of Object-Oriented Programming.

Inheritance is a feature in which new classes are created from the existing classes. The new class created is called “derived class” or “child class” and the existing class is known as the “base class” or “parent class”.

The derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own. These new features in the derived class will not affect the base class.

Syntax:

```
class <derived_class_name> : <access-specifier> <base_class_name>
{
    //body
}
```

Modes of Inheritance: There are 3 modes of inheritance.

1. **Public Mode:** If we derive a subclass from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in the derived class.

2. **Protected Mode:** If we derive a subclass from a Protected base class. Then both public members and protected members of the base class will become protected in the derived class.

3. **Private Mode:** If we derive a subclass from a Private base class. Then both public members and protected members of the base class will become Private in the derived class.

The below table summarizes the above three modes and shows the access specifier of the members of the base class in the subclass when derived in public, protected and private modes:

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Sample Question:

1. Explain the behavior of public and private members when a base class is inherited publicly by the derived class.[Sp-24]

2. How do the properties of the following two derived class differ? [Sp-23]

Class D1: private B{}

Class D2: public B{}

Types of Inheritance in C++

- 1. Single Inheritance:** In Single Inheritance, one class is derived from another class.
- It represents a form of inheritance where there is only one base and derived class.



Syntax:

```
class subclass_name : access_mode <base_class>
{
    // body of subclass
};
```

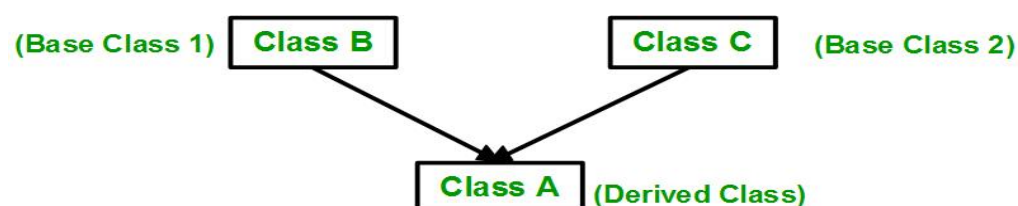
C++ program to explain Single inheritance

```
#include<iostream>
using namespace std;
// base class
class Vehicle {
public:
    Vehicle()
    {
        cout << "This is a Vehicle\n";
    }
};
// sub class derived from a single base classes
class Car : public Vehicle {
    //body of this class
};
int main()
{
    Car obj;
    return 0;
}
```

Output

This is a Vehicle

- 2. Multiple Inheritance:** Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class**.



Syntax:

```
class subclass_name : access_mode <base_class1>, access_mode <base_class2>, ....
{
    // body of subclass
};
```

C++ program to explain multiple inheritance

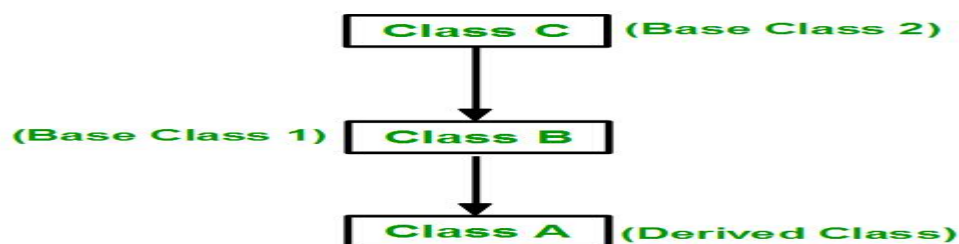
```
#include <iostream>
using namespace std;
// first base class
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle\n";
    }
};
// second base class
class FourWheeler {
public:
    FourWheeler()
    {
        cout << "This is a 4 wheeler Vehicle\n";
    }
};
// sub class derived from two base classes
class Car : public Vehicle, public FourWheeler {
};
int main()
{
    Car obj;
    return 0;
}
```

Output

This is a Vehicle

This is a 4 wheeler Vehicle

3. Multilevel Inheritance: In this type of inheritance, a derived class is created from another derived class.



C++ program to implement Multilevel Inheritance

```
#include <iostream>
using namespace std;
// base class
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle\n";
    }
};
// first sub_class derived from class vehicle
class fourWheeler : public Vehicle {
public:
    fourWheeler()
    {
        cout << "Objects with 4 wheels are vehicles\n";
    }
};
// sub class derived from the derived base class fourWheeler
class Car : public fourWheeler {
public:
    Car() {
        cout << "Car has 4 Wheels\n"; }
};
```

```
int main()
{
    Car obj;
    return 0;
}
```

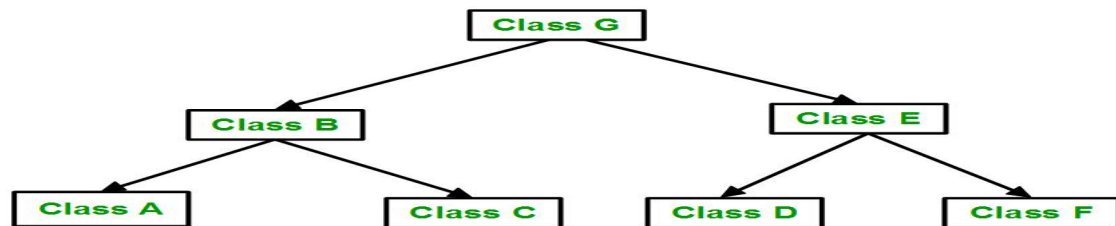
Output:

This is a Vehicle

Objects with 4 wheels are vehicles

Car has 4 Wheels

4. Hierarchical Inheritance: In this type of inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.



C++ program to implement Hierarchical Inheritance

```
#include <iostream>

using namespace std;

// base class
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle\n";
    }
};

// first sub class
class Car : public Vehicle {
};

// second sub class
class Bus : public Vehicle {
};

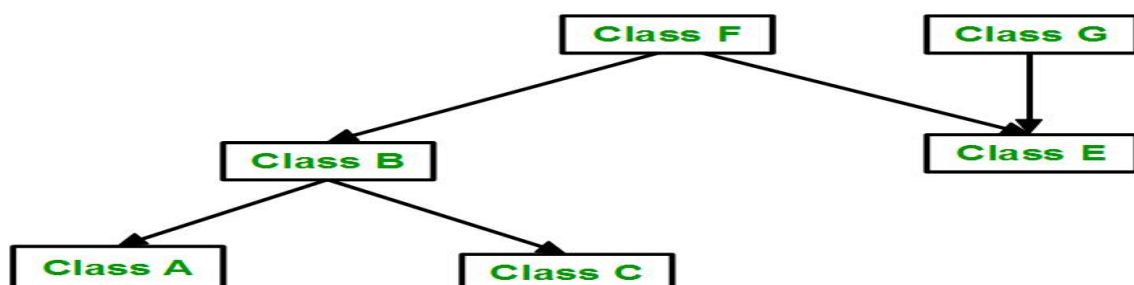
int main()
{
    Car obj1;
    Bus obj2;
    return 0;
}
```

Output

This is a Vehicle

This is a Vehicle

5. Hybrid (Virtual) Inheritance: Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance. Below image shows the combination of hierarchical and multiple inheritances:



C++ program to implement Hybrid Inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};
class Fare {
public:
    Fare() {
        cout << "Fare of Vehicle\n";
    }
};
class Car : public Vehicle {};
class Bus : public Vehicle, public Fare {};
int main()
{
    Bus obj2;
    return 0;
}
```

Output

This is a Vehicle

Fare of Vehicle

Sample Question

1. Write a program to implement multilevel inheritance.[Sp-23]

2.

Create a base class called vehicle that stores the number of wheels a vehicle has and the range of vehicle. Create a derived class called car that inherits vehicle and also store the number of passengers. Then, create a derived class called truck that inherits vehicle and also stores the information of load limit. Display the information

3.

Create a class hierarchy for a bookstore's inventory system with a base class **Item (title and price) and two subclasses, **Book** (author, genre, ISBN) and **CD** (artist, genre, duration). Subclass further with Fiction and Non-Fiction for books and Rock and Pop for CDs, adding category-specific data. Demonstrate the use of this hierarchy by creating and displaying sample instances of various books and CDs, showcasing their details.**

4.

We want to store the information of different vehicles. Create a class named **Vehicle** with two data member named mileage and price. Create its two subclasses **Car** with data members to store ownership cost, warranty (by years), seating capacity and fuel type (diesel or petrol). **Bike** with data members to store the number of cylinders, number of gears, cooling type (air, liquid or oil), wheel type (alloys or spokes) and fuel tank size (in inches). Make another two subclasses **Audi** and **Ford** of **Car**, each having a data member to store the model type. Next, make two subclasses **Bajaj** and **TVS**, each having a data member to store the make-type. Now, store and print the information of an Audi and a Ford car (i.e., model type, ownership cost, warranty, seating capacity, fuel type, mileage and price.) Do the same for a Bajaj and a TVS bike.

Key Feature of Hybrid inheritance

- Hybrid inheritance is also known as *Virtual Inheritance*.
- It is a combination of two or more inheritance.
- In hybrid inheritance, when derived class have multiple paths to a base class, a diamond problem occurs. It will result in duplicate inherited members of the base class.
- To avoid this problem easily, we use **Virtual Base Class**. In this case, derived classes should inherit base class by using Virtual Inheritance

Virtual Base Class

Suppose we have two derived classes B and C that have a common base class A, and we also have another class D that inherits from B and C. We can declare the base class A as *virtual* to ensure that B and C share the same sub object of A.

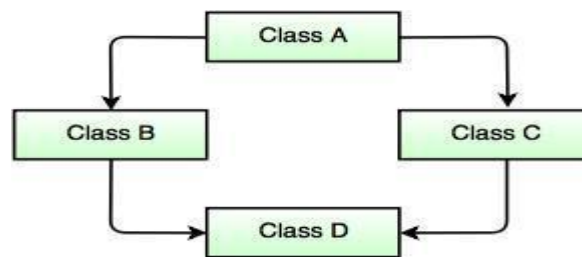


Fig. Hybrid Inheritance

Diamond Problem Solution

```

class A
{ public:
    void eat ()
    {
        cout<<"Please eat."<<endl;
    }
};
class B: virtual public A
{
public:
    void donteat ()
    {
        cout<<"Please DON'T eat!"<<endl;
    }
};
class C: virtual public A
{
};
class D: public B, C
{
};
  
```

```
int main() {
    D obd;
    obd.eat();
    obd.donteat();
}
```

Please eat.
Please DON'T eat!

Sample Question:

1. What is virtual base class? Explain with writing a program[Aut-22]
2. Describe the diamond problem in the context of multiple inheritance in c++.How can it be resolved using virtual inheritance? provide an example[Aut-23]

Ambiguity in Multiple Inheritance

When Two base classes have functions with the same name, while a class derived from both base classes. How do objects of the derived class access the correct base class function ? The name of the function alone is insufficient, since the compiler can't figure out which of the two functions is meant.

Solving Ambiguity in Multiple Inheritance

```
class A
{
    public:
    void show() { cout << "Class A\n"; }
};
class B
{
    public:
    void show() { cout << "Class B\n"; }
};
class C : public A, public B
{
};
int main()
{
    C objC; //object of class C
    // objC.show(); //ambiguous--will not compile
    objC.A::show(); //OK
    objC.B::show(); //OK
    return 0;
}
```

Class A
Class B

The problem is resolved using the scope-resolution operator to specify the class in which the function lies. Thus

objC.A::show(); refers to the version of show() that's in the A class,

objC.B::show(); refers to the function in the B class

Sample Question:

Is there any error in this code? If yes, then correct the code. Display the output.

```
class A
{
    public:
    void cheers()
    {
        cout<<"Class A: Hip-hip-hooray";
    }
};
class B
{
    public:
    void cheers()
    {
        cout<<"Class B: Hip-hip-hooray";
    }
};
class C:public A, public B
{
};
int main()
{
    C obc;
    obc.chears();
}
```

Constructors, Destructors, and Inheritance :

It is possible for the base class, the derived class, or both to have constructor and/or destructor functions. When a base class and a derived class both have constructor and destructor functions, the constructor functions are executed in order of derivation. The destructors functions are executed in reverse order. That is, the base class constructor is executed before the constructor in the derived class. In Case of destructor functions, the derived class's destructor is executed before the base class's destructor.

Here is a short program that illustrates when base class and derived class constructor and destructor functions are executed:

```
# include <iostream >
using namespace std;
class base
{
    public :
    base () { cout << " Constructing base class \n"; }
    ~ base () { cout << " Destructing base class \n"; }
};
class derived
: public base
{
    public :
    derived () { cout << " Constructing derived class \n"; }
    ~ derived () { cout << " Destructing derived class \n"; }
```

Sheet for only classes but for exam book is mandatory

Prepared By: Muhammad Nazim Uddin, Lecturer (Adjunct), CSE, IIUC

```
};
int main ()
{
    derived o;
    return 0;
}
```

This program displays the following output:

```
Constructing base class
Constructing derived class
Destructing derived class
Destructing base class
```

Constructors in derived classes

```
class Base
{
    public:
    Base()
    { cout<<"Base"<<endl; }
    Base(int a)
    { cout<<"From Base: value of a = "<<a;}
};

class Derived : public Base
{
    public:
    Derived():Base()
    {
        cout<<"Derived"<<endl;
    }
    Derived(int d):Base(d)
    { }
};

int main()
{
    Derived obD, obD2(10);

    return 0;
}
```

```
Base
Derived
From Base: value of a = 10
```

Sample Question:

1. How to invoke Base class parameterized constructor inside derived class parameterized constructor? Give example.[Sp-23]

Nesting of Classes

- A nested class is a class which is declared in another enclosing class.
- A nested class is a member and as such has the same access rights as any other member.
- A *nested class* is declared within the scope of another class.

```

class A {
public:
    class B {
    private:
        int num;
    public:
        void getdata(int n) {
            num = n;
        }
        void putdata() {
            cout<<"The number is "<<num;
        }
    };
};

int main() {
    cout<<"Nested classes in C++"<< endl;
    A :: B obj;
    obj.getdata(9);
    obj.putdata();
    return 0;
}

```

Output:

```

Nested classes in C++
The number is 9

```

Previous Question Answer:

Q1. How to invoke Base class parameterized constructor inside derived class parameterized constructor? Give example.

Answer:

```

#include <iostream>
using namespace std;
class base
{
    int i;
public:
    base (int n)
    {
        cout << " Constructing base class \n";
        i = n;
    }
    ~base () { cout << " Destructing base class \n"; }
    void showi () { cout << i << '\n'; }
};

class derived : public base
{
    int j;
public:
    derived (int n) : base (n) // pass arg to base class .
    {
        cout << " Constructing derived class \n";
        j = n;
    }
}

```

```

~ derived () { cout << " Destructing derived class \n"; }
void showj () { cout << j << '\n'; }
};
int main () {
    derived o (10);
    o. showi ();
    o. showj ();
    return 0;
}

```

Q2. write the output of this code

```

#include<bits/stdc++.h>
using namespace std;
class P{
public:
    void print(){
        cout<<"inside P";
    }
};

class Q:public P{
public:
    void print(){cout<<"inside Q";
    }
};
class R : public Q{
};

int main(){

    R r;
    r.print();
    return 0;
}

```

Output:

The output of the code will be
"inside Q"

Explanation:

The code defines three classes - P, Q, and R. Class P has a public member function print(), which outputs the text "inside P". Class Q is a derived class of P and overrides the print() function to output "inside Q". Class R is a derived class of Q but doesn't define its own print() function, so it inherits the one from Q

In the main() function, an object of class R is created, and its print() function is called. Since R doesn't define its own print() function, it uses the one from Q, which outputs "inside Q". Therefore, the output of the program is "inside Q"

c) Is there any error in this code? If yes, then correct the code. Display the output.

```
class A
{
    public:
    void cheers()
    {
        cout<<"Class A: Hip-hip-hooray";
    }
};
class B
{
    public:
    void cheers()
    {
        cout<<"Class B: Hip-hip-hooray";
    }
};
class C:public A, public B
{
};
int main()
{
    C obc;
    obc.cheers();
}
```

Correct Answer:

```
Int main()
{
    V obc;
    obc.A::cheers();
    obc.B::cheers();
}
```

Output:

Class A: Hip-hip-hooray
Class B: Hip-hip-hooray

d) Create a base class called vehicle that stores the number of wheels a vehicle has and the range of vehicle. Create a derived class called car that inherits vehicle and also store the number of passengers. Then, create a derived class called truck that inherits vehicle and also stores the information of load limit. Display the information

Answer:

```
#include <iostream>
using namespace std;
class vehicle{
    public:
    int wheels, speed;
    void getvehicle(){
        cout<<"wheels = " << wheels << endl;
        cout<<"speed = " << speed << endl;
    }
};
class car:public vehicle{
    public:
```

```

    int passengers;
    void getcar(){
        cout<<"Passengers = " << passengers << endl;
    }
};

class truck: public vehicle{
    public:
    int load_lim;
    void gettruck(){
        cout<<"Load Limit = " << load_lim << endl;
    }
};

int main() {
    car car_obj;
    truck truck_obj;
    car_obj.wheels = 4;
    car_obj.speed = 180;
    car_obj.passengers = 5;
    truck_obj.wheels = 6;
    truck_obj.speed = 90;
    truck_obj.load_lim = 500;
    car_obj.getvehicle();
    car_obj.getcar();
    truck_obj.getvehicle();
    truck_obj.gettruck();
    return 0;
}

```

Prepared By
Muhammad Nazim Uddin,
 Lecturer(Adjunct), CSE, IIUC